MODULE 5

1. Worst-Case Boundary Value Analysis for the Next Date Function

Problem Statement

Analyse the effectiveness of Worst-Case Boundary Value Analysis (BVA) testing for the Next Date Function with the following input ranges:

- Month: $1 \leq \text{Month} \leq 12$
- Day: $1 \leq \text{Day} \leq 31$
- Year: $1812 \leq \text{Year} \leq 2025$

Boundary Value Analysis

Boundary Value Analysis (BVA) is a black-box testing technique that tests values at the boundaries of input domains, since errors are more likely to occur near extreme values. Worst-Case BVA tests all combinations of boundary values for every input variable simultaneously, providing more thorough coverage than standard BVA, which assumes only one variable is at a boundary at a time.

Boundary Values

Worst-Case BVA uses five values for each variable: minimum, minimum + 1, nominal (midpoint), maximum − 1, and maximum.

Variable	Min	Min + 1	Nominal	Max − 1	Max
Month	1	2	6	11	12
Day	1	2	16	30	31
Year	1812	1813	1919	2024	2025

Nominal values are calculated as the midpoint of each range: Month (1 + 12) / 2 = 6, Day (1 + 31) / 2 = 16, Year (1812 + 2025) / 2 = 1919.

Total Test Cases

- Regular BVA: $4n + 1 = 4 \times 3 + 1 = 13$ test cases
- Worst-Case BVA: $5^3 = 125$ test cases

Sample Test Cases

TC#	Month	Day	Year	Expected Output
001	6	1	1919	6/2/1919
002	6	2	1919	6/3/1919
003	6	16	1919	6/17/1919
004	6	30	1919	7/1/1919
005	6	31	1919	Invalid date
006	12	31	2024	1/1/2025
007	12	31	2025	Invalid (beyond range)
008	2	28	1919	3/1/1919
009	2	31	1919	Invalid date
010	1	31	1812	2/1/1812

Only 10 sample test cases are shown for illustration; the complete Worst-Case BVA test suite contains 125 test cases.

Effectiveness Analysis

Strengths

- Tests all combinations of extreme values simultaneously, catching defects that standard BVA may miss.

- Detects month-end transitions, such as moving from the 30th or 31st day to the next month.
- Verifies year rollover (e.g., 31/12/2024 → 1/1/2025).
- Detects invalid dates such as February 31 and June 31.
- Provides good coverage of boundary years (1812, 1813, 1919, 2024, 2025).

Weaknesses

- Requires 125 test cases, making execution costly and time-consuming.
- Generates many invalid input combinations, resulting in unnecessary testing effort.
- Does not explicitly focus on leap-year scenarios involving February 29.
- May miss important February-specific cases unless values such as 28 or 29 are intentionally included.

---

2. Weak Normal, Strong Normal, and Weak Robust Equivalence Class Testing for the Triangle Problem

Problem Statement

Examine Weak Normal, Strong Normal, and Weak Robust Equivalence Class Testing for the Triangle Problem with the following constraints:

- $1 \leq a \leq 200$
- $1 \leq b \leq 200$
- $1 \leq c \leq 200$

Equivalence Class Testing

Equivalence Class Testing (ECT) is a black-box testing technique that divides the input domain into equivalence classes, where all values in a class are expected to be processed similarly. Instead of testing every possible input, one representative value from each class is selected.

Equivalence Classes

Valid Equivalence Classes (Normal)

Class	Condition	Example
EC1	Equilateral (a = b = c)	(5, 5, 5)
EC2	Isosceles (exactly two sides equal)	(5, 5, 8)
EC3	Scalene (all sides different and form a valid triangle)	(3, 4, 5)
EC4	Not a Triangle (sum of any two sides ≤ third side)	(1, 2, 4)

Invalid Equivalence Classes (Robust)

Class	Condition	Example
EC5	a < 1	a = 0
EC6	a > 200	a = 201
EC7	b < 1	b = 0
EC8	b > 200	b = 201
EC9	c < 1	c = 0
EC10	c > 200	c = 201

Weak Normal Equivalence Class Testing

Select one test case from each valid equivalence class only; invalid classes are not tested.

Total test cases = 4

TC#	a	b	c	EC Covered	Expected Output
TC1	5	5	5	EC1	Equilateral
TC2	5	5	8	EC2	Isosceles
TC3	3	4	5	EC3	Scalene
TC4	1	2	4	EC4	Not a Triangle

Strong Normal Equivalence Class Testing

Test all possible combinations of valid equivalence classes. Since EC1–EC4 are mutually exclusive (a triangle can belong to only one category at a time), each valid class is tested once, so the total is the same as Weak Normal.

Total test cases = 4

TC#	a	b	c	EC Covered	Expected Output
TC1	10	10	10	EC1	Equilateral
TC2	10	10	15	EC2	Isosceles
TC3	6	8	10	EC3	Scalene
TC4	2	3	10	EC4	Not a Triangle

Weak Robust Equivalence Class Testing

Test all valid equivalence classes (one representative each) and one invalid equivalence class at a time, while keeping the remaining inputs within valid ranges.

Total test cases = 4 (valid) + 6 (invalid) = 10

TC#	a	b	c	EC Covered	Expected Output
TC1	5	5	5	EC1	Equilateral
TC2	5	5	8	EC2	Isosceles
TC3	3	4	5	EC3	Scalene
TC4	1	2	4	EC4	Not a Triangle
TC5	0	5	5	EC5 (a < 1)	Invalid input
TC6	201	5	5	EC6 (a > 200)	Invalid input
TC7	5	0	5	EC7 (b < 1)	Invalid input
TC8	5	201	5	EC8 (b > 200)	Invalid input
TC9	5	5	0	EC9 (c < 1)	Invalid input
TC10	5	5	201	EC10 (c > 200)	Invalid input

3. Commission Problem Using Boundary Value Testing

Commission Problem

The Commission Problem calculates the commission earned by a salesperson based on the number of locks, stocks, and barrels sold.

Sales = (45 × Locks) + (30 × Stocks) + (25 × Barrels)

Commission rules:

- 10% on sales up to $1000
- 15% on the next $800 (i.e., from $1001 to $1800)
- 20% on sales above $1800

Boundary Value Testing

Boundary Value Testing (BVT) is a black-box testing technique that focuses on testing values at or near the boundaries of input or output ranges, where defects are most likely to occur. In the Commission Problem, BVT is applied to the output range (sales amount) because commission rates change at specific thresholds, particularly at $1000. The selected test cases cover the minimum valid value, just below the boundary, exactly at the boundary, and just above the boundary.

Input Ranges

Item	Valid Range
Locks	1 to 70
Stocks	1 to 80
Barrels	1 to 90

The minimum sales amount is $100, representing one complete rifle.

Output Boundary Value Test Cases

Case #	Locks	Stocks	Barrels	Sales ($)	Commission ($)	Comments
1	1	1	1	100	10.00	Minimum
2	10	10	9	975	97.50	Border –
3	10	9	10	970	97.00	Border –
4	9	10	10	955	95.50	Border –
5	10	10	10	1000	100.00	Boundary
6	10	10	11	1025	103.75	Border +
7	10	11	10	1030	104.50	Border +
8	11	10	10	1045	106.75	Border +

Examination of Results

Case 1: Minimum Value

Tests the lowest valid sale amount ($100). Commission is correctly calculated as 10% = $10, verifying that the minimum sales requirement is handled correctly.

Cases 2–4: Just Below the Boundary

Sales values range from $955 to $975, all below the $1000 threshold. The commission remains at 10%, confirming that the program does not prematurely apply the 15% rate.

Case 5: At the Boundary

Sales amount is exactly $1000. Commission = 10% × 1000 = $100, validating behavior at the exact transition point.

Cases 6–8: Just Above the Boundary

Sales values range from $1025 to $1045, just above $1000. The program correctly applies 10% on the first $1000 and 15% on the amount exceeding $1000.

For example, Case 6 (Sales = $1025): Commission = 10% of $1000 + 15% of $25 = $100 + $3.75 = $103.75, confirming the calculation is correct immediately after crossing the boundary.

---

4. Decision Table for the Email Validation System

Decision Table

A Decision Table is a technique used to document and represent logical conditions and their corresponding actions. It helps test all possible combinations of input conditions, ensures no combination is overlooked, and defines expected system behavior for every scenario. A decision table consists of two main parts: conditions (input conditions or criteria) and actions (expected outputs or system responses).

Input Conditions

The system accepts two inputs:

- Email: can be Blank, Invalid, or Valid
- Password: can be Blank, Invalid, or Valid

Total combinations = 3 × 3 = 9

Decision Table

Rule	1	2	3	4	5	6	7
Email	Blank	Blank	Blank	Invalid	Invalid	Invalid	Valid
Password	Blank	Invalid	Valid	Blank	Invalid	Valid	Blank
Expected Result	Error: Please enter email	Error: Please enter email	Error: Please enter email	Error: Please enter a valid email	Login failed	Error: Please enter a valid email	Error: Ple enter pas
Page Displayed	Login Page	Login Page	Login Page	Login Page	Login Page	Login Page	Login Pag

Classification of Test Cases

Valid Combination

Rule	Email	Password	Expected Result
9	Valid	Valid	Login successful (Home Page displayed)

Invalid Combinations

- Rules 1–3 (Email = Blank, Password = Blank/Invalid/Valid): "Please enter email."
- Rules 4 and 6 (Email = Invalid, Password = Blank or Valid): "Please enter a valid email."
- Rule 7 (Email = Valid, Password = Blank): "Please enter password."
- Rules 5 and 8 (Rule 5: Email = Invalid, Password = Invalid; Rule 8: Email = Valid, Password = Invalid): "Login failed."

---

5. Decision Table-Based Testing for the Next Date Problem

Input Conditions

The Next Date function accepts three inputs, each classified as Valid or Invalid:

- C1: $1 \leq \text{Month} \leq 12$
- C2: $1 \leq \text{Day} \leq 31$
- C3: $1812 \leq \text{Year} \leq 2012$

Total combinations = $2 \times 2 \times 2 = 8$

Decision Table

Conditions	R1	R2	R3	R4	R5	R6	R7
Month Valid	T	T	T	T	F	F	F
Day Valid	T	T	F	F	T	T	F
Year Valid	T	F	T	F	T	F	T
Expected Result	Next Date Calculated	Invalid Input	Invalid Input	Invalid Input	Invalid Input	Invalid Input	Invalid Ir
Action	Display Next Date	Error Message	Error Message	Error Message	Error Message	Error Message	Error Mes

T = True, F = False

Classification of Test Cases

Valid Combination

Rule	Month	Day	Year	Expected Result
R1	Valid	Valid	Valid	Next Date is calculated and displayed

Invalid Combinations

Rule	Month	Day	Year	Expected Result
R2	Valid	Valid	Invalid	Invalid Input
R3	Valid	Invalid	Valid	Invalid Input
R4	Valid	Invalid	Invalid	Invalid Input
R5	Invalid	Valid	Valid	Invalid Input
R6	Invalid	Valid	Invalid	Invalid Input
R7	Invalid	Invalid	Valid	Invalid Input
R8	Invalid	Invalid	Invalid	Invalid Input

Sample Test Cases

Test Case	Input (Month, Day, Year)	Expected Output
TC1	(6, 15, 2000)	16/06/2000
TC2	(12, 31, 2000)	01/01/2001
TC3	(2, 28, 2000)	29/02/2000
TC4	(2, 29, 2000)	01/03/2000
TC5	(13, 15, 2000)	Invalid Input
TC6	(6, 32, 2000)	Invalid Input
TC7	(6, 15, 2013)	Invalid Input
TC8	(13, 32, 2013)	Invalid Input

6. Single Fault vs. Multiple Fault Assumptions in Boundary Value and Equivalence Class Testing

Boundary Value Testing (BVT) and Equivalence Class Testing (ECT) are based on assumptions about how faults occur in a program. The two common assumptions are the Single Fault Assumption and the Multiple Fault Assumption, which help in selecting effective test cases.

Single Fault Assumption

The Single Fault Assumption assumes that only one input variable contains a fault at a time, while all other input variables remain at their normal (nominal) values.

Characteristics:

- Only one variable is tested at its boundary value; all others are kept at nominal values.
- Commonly used in basic Boundary Value Analysis.
- Generates fewer test cases, reducing testing effort and execution time.

Example (Next Date Problem): Month = 1 (minimum boundary value), Day = 15 (nominal), Year = 2000 (nominal) — only Month is tested at its boundary while the other variables remain normal.

Multiple Fault Assumption

The Multiple Fault Assumption assumes that more than one input variable may contain faults simultaneously.

Characteristics:

- Multiple variables are tested together at their boundary values, considering combinations of extreme values.
- Used in Worst-Case Boundary Value Testing.
- Generates a larger number of test cases and provides higher test coverage.

Example (Next Date Problem): Month = 1, Day = 1, Year = 1812 — all three variables are tested at their minimum boundary values simultaneously.

Relationship with Boundary Value Testing

Boundary Value Testing focuses on the minimum value, just above minimum, nominal value, just below maximum, and maximum value.

- Basic Boundary Value Testing follows the Single Fault Assumption, where one variable is tested at a boundary while the remaining variables are kept at nominal values.
- Worst-Case Boundary Value Testing follows the Multiple Fault Assumption, where multiple variables are tested together at their boundary values.

Relationship with Equivalence Class Testing

Equivalence Class Testing divides the input domain into valid and invalid equivalence classes and selects one representative value from each class, rather than focusing only on boundary values.

For the Next Date Problem:

- Valid classes: Month = 1 to 12, Day = 1 to 31, Year = 1812 to 2012
- Invalid classes: Month < 1 or Month > 12, Day < 1 or Day > 31, Year < 1812 or Year > 2012

Comparison

Feature	Single Fault Assumption	Multiple Fault Assumption
Basic Idea	Assumes only one input variable contains a fault at a time	Assumes multiple input variables may contain faults simultaneously
Variables Tested	One variable at a boundary; others remain nominal	Multiple variables tested together at boundary values
Test Case Count	Fewer	More
Coverage	Moderate	Higher
Testing Effort	Lower	Higher
Common Usage	Basic Boundary Value Testing	Worst-Case Boundary Value Testing

7. Weak Normal, Strong Normal, and Weak Robust Equivalence Class Testing for the Next Date Problem

Problem Conditions

The Next Date Problem has the following input constraints:

- $1 \leq \text{Month} \leq 12$
- $1 \leq \text{Day} \leq 31$
- $1812 \leq \text{Year} \leq 2012$

Equivalence Classes

Valid Equivalence Classes

Class	Condition
M1	$1 \leq \text{Month} \leq 12$
D1	$1 \leq \text{Day} \leq 31$
Y1	$1812 \leq \text{Year} \leq 2012$

Invalid Equivalence Classes

Class	Condition
M2	Month < 1
M3	Month > 12
D2	Day < 1
D3	Day > 31
Y2	Year < 1812
Y3	Year > 2012

Weak Normal Equivalence Class Testing

Select one representative value from each valid equivalence class; only valid inputs are tested.

Total test cases = 4

TC#	Month	Day	Year	Expected Output
TC1	6	15	2000	16/06/2000
TC2	12	31	2000	01/01/2001
TC3	2	28	2001	01/03/2001
TC4	2	29	2000	01/03/2000

These cases represent a normal date, a year transition, February in a non-leap year, and February in a leap year.

Strong Normal Equivalence Class Testing

Test all important valid combinations of equivalence classes to achieve broader coverage while remaining within valid input ranges.

Total test cases = 8

TC#	Month	Day	Year	Expected Output
TC1	6	15	2000	16/06/2000
TC2	1	1	1812	02/01/1812
TC3	12	31	1812	01/01/1813
TC4	12	31	2012	01/01/2013
TC5	2	28	2001	01/03/2001
TC6	2	28	2000	29/02/2000
TC7	2	29	2000	01/03/2000
TC8	11	30	2000	01/12/2000

These cases cover normal dates, the beginning and end of the valid range, leap-year and non-leap-year behavior, and month/year transitions.

Weak Robust Equivalence Class Testing

Test all valid equivalence classes along with one invalid equivalence class at a time, while keeping the remaining inputs valid.

Total test cases = 10

TC#	Month	Day	Year	Expected Output
TC1	6	15	2000	16/06/2000
TC2	12	31	2000	01/01/2001
TC3	2	28	2001	01/03/2001
TC4	2	29	2000	01/03/2000
TC5	0	15	2000	Invalid Input (M2: Month < 1)
TC6	13	15	2000	Invalid Input (M3: Month > 12)
TC7	6	0	2000	Invalid Input (D2: Day < 1)
TC8	6	32	2000	Invalid Input (D3: Day > 31)
TC9	6	15	1811	Invalid Input (Y2: Year < 1812)
TC10	6	15	2013	Invalid Input (Y3: Year > 2012)

8. Worst-Case Boundary Value Analysis for the Triangle Problem

Input Ranges

- $1 \le a \le 200$
- $1 \le b \le 200$
- $1 \le c \le 200$

Worst-Case Boundary Value Analysis tests all input variables at their boundary values simultaneously. Each variable is assigned five representative values: minimum, minimum + 1, nominal, maximum − 1, and maximum.

Boundary Values

Variable	Min	Min + 1	Nominal	Max − 1	Max
a	1	2	100	199	200
b	1	2	100	199	200
c	1	2	100	199	200

Nominal value = (1 + 200) / 2 = 100

Total Test Cases

Each of the three variables has 5 boundary values: Total = $5^3 = 125$

For comparison, Regular Boundary Value Analysis requires 13 test cases.

Sample Test Cases

TC#	a	b	c	Expected Output
001	100	100	1	Isosceles
002	100	100	2	Isosceles
003	100	100	100	Equilateral
004	100	100	199	Isosceles
005	100	100	200	Not a Triangle
006	1	1	1	Equilateral
007	2	2	2	Equilateral
008	199	199	199	Equilateral
009	200	200	200	Equilateral
010	1	100	200	Not a Triangle

Only a few sample test cases are shown for illustration; the complete Worst-Case Boundary Value Analysis requires 125 test cases.

Effectiveness Analysis

Strengths

- Tests all combinations of boundary values for a, b, and c, providing maximum coverage compared to normal Boundary Value Analysis.
- Helps detect faults that occur when multiple inputs simultaneously take extreme values.
- Effectively identifies errors related to the triangle inequality conditions.
- Increases the likelihood of discovering hidden defects that may not be detected by testing one variable at a time.

Weaknesses

- Generates a large number of test cases (125), making execution time-consuming and resource-intensive.
- Many test cases may produce identical outputs, leading to redundant testing.
- Some combinations provide little or no additional fault-detection benefit.
- May not be suitable when testing time or resources are limited.

---

9. Triangle Problem Using Decision Table Testing

Input Conditions

The Triangle Program takes three inputs — sides a, b, and c — with the following conditions:

- C1: Is a = b?

- C2: Is b = c?
- C3: Is a = c?
- C4: Is the Triangle Inequality satisfied?

A valid triangle must satisfy all of: a < b + c, b < a + c, and c < a + b.

Decision Table

Conditions	R1	R2	R3	R4	R5
a = b	T	T	F	F	–
b = c	T	F	T	F	–
a = c	T	F	F	F	–
Triangle Valid	T	T	T	T	F
Expected Result	Equilateral	Isosceles	Isosceles	Scalene	Not a Triangle
Action	Display Equilateral	Display Isosceles	Display Isosceles	Display Scalene	Display Error

T = True, F = False

Classification of Test Cases

Valid Combinations

- R1 (a = b = c, Triangle Valid): Equilateral Triangle
- R2 (a = b, a ≠ c, Triangle Valid): Isosceles Triangle
- R3 (b = c, a ≠ b, Triangle Valid): Isosceles Triangle
- R4 (a ≠ b, b ≠ c, a ≠ c, Triangle Valid): Scalene Triangle

Invalid Combination

- R5 (Triangle Inequality not satisfied): Not a Triangle (Display Error)